

FUT MANAGER — GAME DESIGN & TECHNICAL DESIGN

Document de conception de référence — Version 1.0

Pitch : jeu navigateur de management/collection football inspiré de FUT et Football Manager, centré sur la composition d'équipe et la tactique pré-match. Aucun contrôle pendant la rencontre : le serveur pré-simule un match déterministe à partir d'une seed, puis le navigateur affiche un replay 2D vue du dessus.

Objectif de ce document : fournir une spécification suffisamment précise pour développer un premier moteur headless, le connecter à un viewer 2D, puis itérer par configuration et simulation massive sans réécrire le cœur du système.

Décisions structurantes déjà validées : 6 statistiques publiques (Vitesse, Tir, Passe, Physique, Technique, Intelligence), PvP asynchrone + bots, collection permanente, formations prédéfinies, 5 remplacements, fatigue intra-match uniquement, synergie de nationalité locale entre voisins tactiques, rôles simples et tactique collective minimale.

Document généré le 20 juillet 2026.

Sommaire

- 1. Vision et principes directeurs
- 2. Périmètre MVP et non-objectifs
- 3. Boucle de jeu et cycle d'un match
- 4. Modèle temporel : 90 minutes affichées, simulation courte
- 5. Modèle de données joueurs et import CSV
- 6. Construction des six statistiques
- 7. Postes, formations et compatibilité hors poste
- 8. Rôles individuels et instructions collectives
- 9. Synergie de nationalité par proximité tactique
- 10. Architecture du moteur de match
- 11. Mouvement et positionnement sans ballon
- 12. IA utilitaire, Intelligence et température de décision
- 13. Résolution des actions et hasard déterministe
- 14. Ballon, possession, duels et transitions
- 15. Tir, gardiens et buts
- 16. Fatigue, blessures, cartons et remplacements
- 17. Replay 2D et format des événements
- 18. Architecture web, serveur et persistance
- 19. Configuration externe et paramètres d'équilibrage
- 20. Tests, télémétrie et équilibrage statistique
- 21. Roadmap d'implémentation
- 22. Critères d'acceptation du MVP
- Annexes : matrices, pseudo-code, schémas TypeScript et paramètres initiaux

1. Vision et principes directeurs

Le jeu doit être compris comme un manager tactique minimaliste, et non comme un simulateur de football réaliste au sens physique. Le plaisir recherché vient de la préparation : choisir les joueurs, les placer, leur attribuer un rôle, exploiter les synergies, puis observer comment ces choix se matérialisent dans une simulation autonome.

1.1 Les cinq piliers

- Préparation avant contrôle : toutes les décisions importantes sont prises avant le coup d'envoi.
- Lisibilité : le joueur doit pouvoir comprendre pourquoi une équipe joue bien ou mal.
- Simplicité systémique : peu de statistiques et peu de paramètres, mais chacun doit avoir un effet identifiable.
- Déterminisme reproductible : même entrée + même seed = même match, indispensable pour le debug et l'équilibrage.
- Paramétrabilité : presque tous les coefficients doivent vivre dans une configuration externe et non dans le code métier.

1.2 Promesse joueur

« Je construis une équipe, je définis son comportement, puis je regarde mes décisions jouer à ma place. »

1.3 Philosophie de simulation

Le moteur ne cherche pas à reproduire toutes les règles, tous les gestes et toutes les subtilités du football professionnel. Il cherche à produire des matchs crédibles, variés, explicables et équilibrables. Une abstraction correcte est préférable à une physique complexe difficile à maîtriser.

2. Périmètre MVP et non-objectifs

2.1 Inclus dans le MVP

- Base de joueurs importée depuis un CSV de type FIFA/EA FC.
- Six statistiques publiques normalisées de 1 à 100.
- Nationalité, poste principal et postes alternatifs.
- Composition de 11 titulaires + banc.
- Formations prédéfinies.
- Rôles individuels : Défensif, Normal, Offensif, Créateur, Pressing.
- Deux instructions collectives : hauteur de bloc et style de construction.
- Synergie de nationalité entre voisins tactiques.
- Match pré-simulé côté serveur avec seed.
- Viewer 2D vue du dessus avec formes géométriques.
- Fatigue intra-match, 5 remplacements, blessures et cartons.
- Replay, accélération $\times 2/\times 4$ et résultat direct.
- PvP asynchrone et bots à terme ; le premier jalon reste le moteur headless.

2.2 Hors périmètre initial

- Entraînement, progression de carrière, moral, contrats, agents, salaires, transferts complexes.

- Gestion financière de club, staff, infrastructures ou conférences de presse.
- Intervention tactique en direct pendant un match.
- Physique de ballon réaliste, animations avancées ou graphismes 3D.
- Accumulation longue durée de fatigue.
- Système de blessures multi-semaines ; le modèle initial est volontairement simple.

3. Boucle de jeu et cycle d'un match

3.1 Boucle principale

Étape	Action joueur	Résultat système
1	Sélectionner une composition enregistrée	11 titulaires + banc + formation
2	Attribuer les rôles et tactiques	Paramètres pré-match figés
3	Lancer la rencontre	Création de la seed et du MatchConfig
4	Pré-simulation serveur	Résultat + événements + replay
5	Visionner ou passer	Lecture à x1 / x2 / x4 ou résultat direct
6	Fin de match	Score, statistiques, notifications blessures/cartons
7	Préparer le prochain match	Suspensions/blessures simples appliquées

3.2 Immutabilité pendant le match

Dès le lancement, la tactique est figée. Le coach ne peut pas modifier manuellement son onze ni ses consignes. Les remplacements possibles sont uniquement ceux décrits dans le plan pré-match et exécutés automatiquement par le moteur.

4. Modèle temporel : 90 minutes affichées, simulation courte

Le chrono visible représente toujours un match de 90 minutes, mais le moteur ne simule pas 5 400 secondes. Le temps affiché et le temps logique sont dissociés.

4.1 Valeurs par défaut proposées

Paramètre	Défaut	Commentaire
displayed_match_minutes	90	Chronomètre football affiché
logical_match_seconds	360	Durée logique interne
display_factor	15	1 seconde logique = 15 secondes affichées
ai_decision_interval	1.0 s logique	Une décision haut niveau par joueur
physics_step	0.2 s logique	Mouvement/ballon à 5 Hz
viewer_duration_x1	~6 min réelles	Lecture naturelle du replay

Avec 360 secondes logiques et 22 joueurs, on obtient environ 7 920 décisions IA haut niveau. Les mises à jour de mouvement peuvent être plus fréquentes sans recalculer une décision complète.

4.2 Conversion

```
displayed_seconds = logical_time / logical_match_seconds * (90 * 60)
displayed_minute = floor(displayed_seconds / 60)
```

Toutes ces valeurs sont configurables. Une compétition pourrait afficher 90 minutes tout en simulant 240, 360 ou 600 secondes logiques.

5. Modèle de données joueurs et import CSV

Le moteur ne doit jamais dépendre directement du format exact du CSV d'origine. Un script d'import transforme chaque ligne source en un schéma canonique interne.

5.1 Données minimales conservées

Champ canonique	Usage
id	Identifiant stable interne
name	Affichage
nation	Synergie
primary_position	Compatibilité poste
alternative_positions[]	Compatibilité poste
speed / shooting / passing / physical / technical / intelligence	Six stats principales
source_attributes	Optionnel : audit et recalcul futur

5.2 Attributs sources exploitables

Les datasets FC 26 disponibles contiennent typiquement les colonnes nécessaires : Acceleration, Sprint Speed, Finishing, Shot Power, Long Shots, Volleys, Penalties, Vision, Crossing, Short Passing, Long Passing, Curve, Dribbling, Agility, Balance, Reactions, Ball Control, Composure, Interceptions, Def Awareness, Jumping, Stamina, Strength, Aggression, Position, Alternative positions, Nation, ainsi que les cinq attributs de gardien.

5.3 Pipeline recommandé

```
CSV brut
-> parse + normalisation des noms de colonnes
-> validation [0..100]
-> calcul des 6 stats
-> parsing positions/nationalité
-> écriture players.normalized.json ou table DB
-> rapport d'anomalies (valeurs manquantes, positions inconnues)
```

Le script de transformation doit être autonome, versionné et réexécutable. Les formules sont chargées depuis un fichier de configuration pour permettre des ajustements sans modifier la logique d'import.

6. Construction des six statistiques

Les valeurs ci-dessous sont des formules de départ, conçues pour être simples, monotones et facilement modifiables. Elles ne cherchent pas à reproduire les agrégats officiels EA.

6.1 Joueurs de champ

Stat	Formule initiale recommandée
Vitesse	45% Acceleration + 55% Sprint Speed
Tir	45% Finishing + 25% Shot Power + 15% Long Shots + 10% Volleys + 5% Penalties
Passe	30% Short Passing + 25% Vision + 20% Long Passing + 15% Crossing + 10% Curve
Physique	30% Strength + 25% Stamina + 20% Aggression + 15% Jumping + 10% Balance
Technique	35% Ball Control + 30% Dribbling + 15% Agility + 10% Balance + 10% Composure

Stat	Formule initiale recommandée
Intelligence	25% Reactions + 20% Composure + 20% Vision + 20% max(Positioning, Def Awareness) + 15% Interceptions

Toutes les sorties sont arrondies et clampées entre 1 et 100. Le choix de max(Positioning, Def Awareness) évite de pénaliser un joueur spécialisé : un attaquant peut être très intelligent offensivement sans disposer d'une grande awareness défensive, et inversement.

6.2 Gardiens

Les gardiens conservent les mêmes six statistiques publiques afin de garder une interface uniforme, mais leur calcul utilise prioritairement les attributs GK.

Stat	Formule gardien proposée
Vitesse	50% Acceleration + 30% Sprint Speed + 20% GK Reflexes
Tir	70% GK Kicking + 30% Shot Power
Passe	50% GK Kicking + 25% Short Passing + 25% Long Passing
Physique	35% Strength + 25% Jumping + 20% Stamina + 20% Aggression
Technique	40% GK Handling + 25% Ball Control + 20% GK Diving + 15% Composure
Intelligence	40% GK Positioning + 25% GK Reflexes + 20% Reactions + 15% Composure

6.3 Principe fondamental

La stat Intelligence décide mieux ; elle n'exécute pas mieux. Une passe excellente choisie par un joueur intelligent peut être ratée si sa Passe est faible. Inversement, un joueur très technique mais peu intelligent peut exécuter parfaitement une mauvaise décision.

7. Postes, formations et compatibilité hors poste

7.1 Liste MVP

Postes supportés : GK, LB, CB, RB, CDM, CM, CAM, LM, RM, LW, RW, ST. Les variantes gauche/droite d'un même rôle central ne sont pas des postes distincts.

7.2 Formation = graphe de slots

Une formation contient une liste de slots. Chaque slot possède un poste attendu, une position d'ancrage normalisée (x,y) et une liste de voisins tactiques. Les voisins servent à la synergie de nationalité et peuvent aussi servir à certaines décisions de passe.

```
FormationSlot {
  id: "LCM",
  expectedPosition: "CM",
  anchor: { x: 0.42, y: 0.52 },
  neighbors: ["CDM", "RCM", "LW", "LB"]
}
```

7.3 Compatibilité hors poste

La compatibilité C varie de 0 à 1. Les postes naturels et alternatifs du dataset valent 1. Les autres compatibilités proviennent d'une matrice configurable.

Cas	C recommandé	Effet
Poste naturel / alternatif	1.00	Aucun malus
Très proche	0.90	Petit malus d'Intelligence
Compatible	0.75	Malus tactique visible

Cas	C recommandé	Effet
Éloigné	0.55	Fort malus tactique
Joueur de champ en GK / GK au champ	0.20	Cas extrême

7.4 Application du malus

```

INT_position = Intelligence * C
Passing_position = Passing * (0.90 + 0.10*C)
Technique_position = Technique * (0.90 + 0.10*C)
Shooting_position = Shooting * (0.95 + 0.05*C)

```

Speed et Physical restent inchangés.

Ainsi, un attaquant placé défenseur conserve sa vitesse et sa puissance mais lit beaucoup moins bien les situations défensives.

7.5 Matrice initiale simplifiée

De \ Vers	GK	LB/RB	CB	CDM	CM	CAM	LM/RM	LW/RW	ST
GK	1.00	0.20	0.20	0.20	0.20	0.20	0.20	0.20	0.20
LB/RB	0.20	1.00	0.80	0.80	0.75	0.60	0.90	0.80	0.50
CB	0.20	0.75	1.00	0.90	0.70	0.50	0.55	0.40	0.45
CDM	0.20	0.75	0.90	1.00	0.90	0.75	0.70	0.55	0.50
CM	0.20	0.65	0.65	0.90	1.00	0.90	0.85	0.70	0.65
CAM	0.20	0.50	0.45	0.70	0.90	1.00	0.85	0.85	0.80
LM/RM	0.20	0.85	0.50	0.65	0.85	0.85	1.00	0.95	0.70
LW/RW	0.20	0.70	0.35	0.45	0.70	0.85	0.95	1.00	0.85
ST	0.20	0.40	0.40	0.45	0.60	0.80	0.70	0.85	1.00

8. Rôles individuels et instructions collectives

8.1 Rôles universels

Rôle	Effet IA principal
Défensif	Ancre plus basse, remplacement prioritaire, moins de risque, moins d'appels profonds.
Normal	Poids neutres du poste.
Offensif	Ancre plus haute, davantage d'appels, davantage de prises de risque.
Créateur	Recherche du ballon, priorité aux passes progressives, vision de jeu plus sollicitée.
Pressing	Déclenchement plus fréquent du pressing, rayon de pression accru, coût énergétique supérieur.

Le rôle ne remplace pas l'algorithme du poste : il modifie les poids d'un comportement de base.

8.2 Instructions collectives MVP

Instruction	Valeurs	Conséquence
Hauteur de bloc	Bas / Normal / Haut	Décale les ancres défensives et la ligne de pressing.
Construction	Courte / Équilibrée / Directe	Modifie préférence de distance de passe, progression et risque.

9. Synergie de nationalité par proximité tactique

La synergie ne s'applique pas à toute l'équipe. Elle circule sur le graphe des voisins de formation. Deux joueurs voisins de même nationalité créent un lien de synergie.

9.1 Bonus proposé

Nombre de voisins directs de même nationalité	Bonus Intelligence effective
0	+0
1	+2
2	+4
3 ou plus	+6 (cap)

```
INT_effective =  
clamp(INT_position + synergy_bonus - fatigue_penalty, 1, 100)
```

Ce bonus représente la compréhension mutuelle et la coordination. Il n'augmente ni la vitesse ni la technique brute.

9.2 Pourquoi un graphe

Une nationalité commune entre le gardien et l'attaquant n'a pas la même valeur tactique qu'entre un défenseur central et son latéral. Le graphe rend le système local, lisible et compatible avec toutes les formations.

10. Architecture du moteur de match

10.1 Contrat d'entrée/sortie

```
simulateMatch({  
  homeTeam,  
  awayTeam,  
  homeTactics,  
  awayTactics,  
  matchConfig,  
  seed  
}) -> {  
  result,  
  stats,  
  events,  
  replay,  
  diagnostics  
}
```

10.2 Sous-systèmes

Sous-système	Responsabilité
RNG déterministe	Toutes les décisions aléatoires dérivent de la seed.
Match state	Score, temps, possession, énergie, cartons, blessures.
Formation engine	Ancres et zones de responsabilité.
Player AI	Génération et scoring des actions.
Movement	Déplacement vers une cible et contraintes terrain.
Ball engine	Contrôlé / en transit / libre.
Action resolver	Passe, tir, duel, interception, dribble.
Rules	Buts, sorties, fautes simplifiées, cartons, changements.
Replay recorder	Snapshots compressés + événements.

10.3 États collectifs

- Attaque organisée
- Défense organisée
- Transition offensive
- Transition défensive

L'état d'équipe influe sur la cible spatiale de tous les joueurs. Une transition peut durer quelques secondes logiques avant de rejoindre un état organisé.

11. Mouvement et positionnement sans ballon

11.1 Position cible

Chaque joueur possède une ancre de formation. Sa cible instantanée est calculée par addition de plusieurs offsets.

```
target =
formation_anchor
+ possession_phase_offset
+ role_offset
+ collective_tactic_offset
+ ball_attraction_offset
+ local_spacing_offset
```

11.2 Principes

- Le joueur ne poursuit jamais directement le ballon sauf si son action courante le demande.
- Chaque poste possède une zone de responsabilité.
- L'attraction vers le ballon est plus forte pour les joueurs proches et pour le rôle Pressing.
- Un terme de spacing évite que plusieurs coéquipiers occupent exactement la même zone.
- La Vitesse fixe la vitesse maximale ; l'Intelligence peut réduire le délai de réaction avant changement de cible.

11.3 Mouvement avec ballon

Le porteur se déplace plus lentement qu'un joueur sans ballon. La pénalité dépend de Technique : un joueur très technique conserve une plus grande fraction de sa vitesse maximale.

12. IA utilitaire, Intelligence et température de décision

12.1 Pipeline d'une décision

Étape	Description
1. Perception	Lire positions, ballon, score, temps, pression, énergie.
2. Candidats	Créer uniquement les actions légalement/plausiblement disponibles.
3. Utility	Attribuer un score $U(a)$ à chaque action.
4. Température	Calculer T à partir de l'Intelligence effective.
5. Sampling	Choisir par softmax plutôt que argmax strict.
6. Exécution	Résoudre l'action avec les stats techniques et la difficulté.

12.2 Température

```

I = INT_effective / 100
T = T_min + (1 - I)^gamma * (T_max - T_min)

Valeurs initiales :
T_min = 0.06
T_max = 0.55
gamma = 1.5

```

Un joueur intelligent a une température basse et choisit presque toujours une action parmi les meilleures. Un joueur moins intelligent explore davantage de décisions sous-optimales.

12.3 Softmax

$$P(\text{action}=a) = \exp(U(a)/T) / \sum_b \exp(U(b)/T)$$

12.4 Candidats par contexte

Contexte	Actions principales
Avec ballon	Passer, avancer, dribbler, tirer, dégager, temporiser.
Sans ballon, possession alliée	Se démarquer, appel, soutien, largeur, rester en position.
Sans ballon, possession adverse	Marquer, presser, intercepter, tacler, couvrir, se replacer.

12.5 Exemples d'utilité

```

U_shot =
shot_quality(angle, distance)
* shooting_intent(role, position)
* space_available
* score_context

U_pass =
receiver_value
* progression
* pass_safety
* tactic_preference

U_run =
free_space
* expected_progression
* role_run_weight

```

Les stats du joueur peuvent influencer l'estimation de l'utilité, mais doivent surtout influencer l'exécution. L'Intelligence influence la qualité de sélection via la température.

13. Résolution des actions et hasard déterministe

La « réussite » n'est pas une statistique. Elle émerge d'un tirage pseudo-aléatoire déterministe appliqué à une probabilité calculée.

13.1 Forme générale

```

p_success = sigmoid(
actor_quality
- opponent_quality
- difficulty
- pressure
- fatigue_penalty
+ context_bonus
)

success = seeded_rng.next() < p_success

```

13.2 Séparation décision / exécution

Question	Stat dominante
Est-ce une bonne idée de passer ici ?	Intelligence

Question	Stat dominante
La passe arrive-t-elle correctement ?	Passe + Technique
Le joueur choisit-il le bon moment pour tirer ?	Intelligence
Le tir est-il bien réalisé ?	Tir + Technique
Le joueur se place-t-il bien pour défendre ?	Intelligence
Gagne-t-il le duel ?	Physique + Intelligence + contexte

14. Ballon, possession, duels et transitions

14.1 Trois états du ballon

État	Description
CONTROLLED(playerId)	Un joueur contrôle le ballon.
IN_TRANSIT	Passe, tir ou dégagement avec destination/vitesse.
LOOSE	Ballon libre après duel, interception imparfaite ou contact.

14.2 Ballon libre

Lors d'un contact incertain, le moteur évalue les candidats proches. Le score de récupération combine distance, Vitesse, Physique et Intelligence, puis applique un tirage pondéré.

```

contest_score =
w_dist * proximity
+ w_speed * Speed
+ w_phys * Physical
+ w_int * INT_effective

winner = weighted_seeded_sample(contest_scores)

```

14.3 Transition

Une perte de balle bascule immédiatement l'équipe qui perd le ballon en transition défensive et l'autre en transition offensive. Après un délai configurable ou lorsque la structure est rétablie, l'état devient organisé.

15. Tir, gardiens et buts

15.1 Qualité d'occasion

La qualité d'une occasion est une valeur interne de 0 à 1, analogue à un xG simplifié.

```

shot_quality =
f(distance)
* f(angle)
* (1 - pressure)
* body_control
* optional_one_on_one_bonus

```

15.2 Qualité du tir

```

shot_execution =
0.65 * Shooting
+ 0.20 * Technique
+ 0.15 * INT_effective

```

L'Intelligence a un poids faible ici : elle représente le choix du geste et le sang-froid, pas la qualité mécanique principale.

15.3 Capacité d'arrêt du gardien

```
gk_save_quality =  
0.35 * Technique  
+ 0.35 * INT_effective  
+ 0.15 * Speed  
+ 0.15 * Physical
```

15.4 Résolution

La probabilité de but combine qualité d'occasion, exécution du tireur et qualité d'arrêt. Une mauvaise frappe peut sortir du cadre ; une frappe cadrée peut être arrêtée ou repoussée en ballon libre.

15.5 Rebond

Un arrêt non capté crée un ballon LOOSE près du gardien. La probabilité de capter proprement dépend principalement de Technique et Intelligence du gardien.

16. Fatigue, blessures, cartons et remplacements

16.1 Énergie

Chaque joueur commence le match à 100 d'énergie. L'énergie est entièrement réinitialisée après chaque match.

16.2 Dépense

Action	Coût relatif
Déplacement lent	Très faible
Course	Faible
Sprint / appel profond	Moyen
Pressing	Moyen à élevé
Duel / tacle	Élevé ponctuellement

```
energy_cost =  
base_cost(action)  
* (1.20 - 0.004 * Physical)  
  
effective_stat_multiplier =  
0.75 + 0.25 * sqrt(energy / 100)
```

À énergie nulle, un joueur conserve donc environ 75% de son potentiel effectif au lieu de devenir inutilisable.

16.3 Blessures — règle MVP

- Une blessure peut survenir lors d'un contact ou à forte fatigue.
- Le joueur blessé doit sortir dès le prochain arrêt de jeu.
- S'il reste un remplacement compatible configuré, il est utilisé.
- Sinon l'équipe continue à dix.
- Pour le MVP, une blessure rend le joueur indisponible pour le prochain match uniquement, puis il revient à 100%.

16.4 Cartons — règle MVP

- Un jaune : le joueur reste sur le terrain ; aucune accumulation entre matchs.
- Deux jaunes dans le même match : expulsion immédiate et suspension du prochain match.

- Rouge direct : expulsion immédiate et suspension du prochain match.
- Toute expulsion ne produit qu'un match de suspension dans le modèle initial.
- Les cartons sont inclus dans les notifications de fin de match.

16.5 Plan de remplacements pré-match

Le coach peut préparer jusqu'à cinq remplacements. Les déclencheurs sont évalués automatiquement.

Déclencheur	Exemple
Fatigue	Remplacer ST si énergie < 35% après la 55e minute affichée.
Carton	Remplacer un joueur averti si un remplaçant compatible existe.
Blessure	Priorité absolue.
Minute	Remplacer à partir de la 70e minute.
État du score	Option avancée : si mène / si perd.

16.6 Priorité

1. Blessure
2. Expulsion : pas de remplacement du joueur expulsé
3. Règle explicite du coach
4. Fatigue critique
5. Carton selon règle préconfigurée
6. Fatigue standard

Tous les changements sont appliqués au prochain arrêt logique de jeu afin d'éviter les substitutions en plein mouvement.

17. Replay 2D et format des événements

17.1 Principe

Le navigateur ne simule pas le match. Il lit un replay déjà produit. Cela garantit que résultat direct et visionnage complet correspondent exactement au même match.

17.2 Données minimales

```
Replay {
  version,
  seed,
  logicalDuration,
  displayedDuration,
  frameInterval,
  frames: [
    {
      t,
      ball: {x, y, state, ownerId?},
      players: [{id, x, y, energy, state}]
    }
  ],
  events: [...]
}
```

17.3 Événements

- KICKOFF, HALF_TIME, FULL_TIME
- PASS, SHOT, SAVE, GOAL
- TACKLE, INTERCEPTION, FOUL
- YELLOW_CARD, RED_CARD
- INJURY, SUBSTITUTION

- BALL_OUT, CORNER ou remise en jeu simplifiée si implémentés

17.4 Compression

On peut enregistrer les positions toutes les 0,5 à 1,0 seconde logique, puis interpoler côté client. Les événements critiques portent un timestamp exact.

17.5 Viewer

- Terrain 2D normalisé.
- Cercles/triangles pour joueurs, petit cercle pour ballon.
- Couleurs d'équipe configurables.
- Chrono 0-90 minutes.
- Score, pause, ×1, ×2, ×4, barre de progression.
- Option « résultat direct » sans lecture du replay.

18. Architecture web, serveur et persistance

18.1 Recommandation

TypeScript de bout en bout est un très bon choix initial : logique partagée, typage des objets de match et possibilité d'exécuter le moteur en Node.js.

18.2 Découpage

Package / service	Rôle
@game/shared	Types, enums, validation, configuration.
@game/player-data	Import CSV et calcul des stats.
@game/match-engine	Simulation pure sans dépendance UI/DB.
@game/replay	Sérialisation/compression du replay.
API server	Création de match, persistance, auth ultérieure.
Web client	Composition, tactiques, viewer 2D.

18.3 Propriété critique du moteur

Le package de simulation doit être une fonction pure autant que possible : aucune lecture réseau, aucune horloge système, aucun Math.random(), aucune dépendance au DOM.

18.4 PvP asynchrone

Une composition publiée peut être sérialisée comme un snapshot tactique. L'adversaire lance ensuite un match contre ce snapshot. La présence simultanée des deux joueurs n'est pas nécessaire.

19. Configuration externe et paramètres d'équilibrage

Tout coefficient de game design doit être regroupé dans des fichiers JSON/YAML versionnés.

19.1 Exemple

```
match:
  logicalSeconds: 360
  physicsStep: 0.2
```

```
aiDecisionInterval: 1.0
maxSubstitutions: 5

decision:
temperatureMin: 0.06
temperatureMax: 0.55
temperatureGamma: 1.5

fatigue:
minStatMultiplifier: 0.75

synergy:
intBonusByMatchingNeighbors: [0, 2, 4, 6]
cap: 6

roles:
pressing:
pressRadiusMultiplier: 1.25
fatigueCostMultiplier: 1.30
```

19.2 Versionnage

Chaque match sauvegardé doit référencer un engineVersion et un balanceConfigVersion. Ainsi un ancien replay reste interprétable même après modification des coefficients.

20. Tests, télémétrie et équilibrage statistique

20.1 Tests déterministes

- Même seed + mêmes entrées = résultat, événements et replay identiques.
- Aucun NaN/Infinity.
- Tous les joueurs restent dans les limites du terrain.
- Un seul propriétaire du ballon maximum.
- Le nombre de remplacements ne dépasse jamais cinq.
- Un joueur expulsé ne revient jamais sur le terrain.

20.2 Tests de monotonie

- Augmenter uniquement Tir ne doit pas réduire durablement le taux de conversion.
- Augmenter Passe doit améliorer la réussite des passes à difficulté comparable.
- Augmenter Intelligence doit réduire la fréquence des choix fortement sous-optimaux.
- Augmenter Physique doit améliorer la résistance à la fatigue et les duels.

20.3 Monte Carlo

Le moteur doit pouvoir exécuter des milliers de matchs sans replay visuel complet. On compare les distributions : buts/match, tirs, possession, passes, cartons, blessures, fatigue, avantage domicile éventuel et taux de victoire par formation.

20.4 Diagnostics par décision

En mode debug, chaque décision IA peut produire : candidats, utilities, température, probabilité softmax et action choisie. C'est essentiel pour comprendre un comportement aberrant sans observer uniquement le résultat final.

20.5 Métriques de santé proposées

Métrique	But
Buts par match	Éviter 0-0 permanents ou scores absurdes.
Tirs / tirs cadrés	Calibrer création d'occasions.
Passes tentées / réussies	Calibrer construction.
Possession	Vérifier diversité tactique.
Distance parcourue / énergie finale	Calibrer fatigue.
Actions sous-optimales par niveau d'INT	Valider la température.
Win rate par formation/rôle	Détecter les stratégies dominantes.

21. Roadmap d'implémentation

Phase	Livrable	Critère de sortie
0	Types + config + RNG seedé	Simulation vide parfaitement déterministe.
1	Import CSV + 6 stats	Base joueurs canonique générée.
2	Terrain + formations + mouvement	22 joueurs se déplacent sans ballon.
3	Possession + passes + interceptions	Une équipe peut conserver/progresser.
4	Tirs + gardien + score	Match complet produit un résultat.
5	IA utilitaire + température	Profils d'INT se comportent différemment.
6	Fatigue + subs + blessures + cartons	Cycle de match complet.
7	Replay + viewer 2D	Match visionnable à ×1/×2/×4.
8	Monte Carlo + diagnostics	Équilibrage automatisé.
9	API + compos enregistrées + bots	Premier produit jouable.
10	PvP asynchrone	Défis contre snapshots d'adversaires.

22. Critères d'acceptation du MVP

- Deux équipes complètes peuvent être chargées à partir du modèle canonique.
- Le match se simule intégralement côté serveur avec une seed.
- Une même seed reproduit exactement le même match.
- Le score résulte d'actions simulées et non d'un tirage de score préalable.
- Les six statistiques ont des effets mesurables.
- L'Intelligence influence la qualité de décision via la température.
- Le hors-poste réduit surtout l'Intelligence.
- La synergie nationale locale augmente l'Intelligence effective.
- La fatigue se construit pendant le match et se réinitialise ensuite.
- Jusqu'à cinq substitutions automatiques peuvent avoir lieu selon le plan pré-match.
- Cartons et blessures produisent les conséquences définies.
- Le viewer 2D peut rejouer le match sans recalculer la simulation.
- Le moteur peut simuler au moins plusieurs milliers de matchs en batch pour l'équilibrage.

Annexe A — Pseudo-code de boucle principale

```
function simulateMatch(input):
    rng = SeededRNG(input.seed)
    state = initializeMatch(input)
    recorder = ReplayRecorder()

    while state.logicalTime < config.logicalSeconds:
        updateTeamPhases(state)

        if isDecisionTick(state.logicalTime):
            for player in activePlayers(state):
                perception = perceive(state, player)
                candidates = generateCandidates(player, perception)
                utilities = scoreUtilities(player, candidates, perception)
                temperature = temperatureFromIntelligence(player.effectiveInt)
                action = softmaxSample(candidates, utilities, temperature, rng)
                player.currentAction = action

            resolveOngoingActions(state, rng)
            integrateMovement(state, config.physicsStep)
            integrateBall(state, config.physicsStep)
            resolveContactsAndLooseBall(state, rng)
            applyRules(state, rng)
            evaluateAutomaticSubstitutions(state)

            recorder.captureIfNeeded(state)
            state.logicalTime += config.physicsStep

    return finalize(state, recorder)
```

Annexe B — Schémas TypeScript indicatifs

```
type Position =
    | 'GK' | 'LB' | 'CB' | 'RB'
    | 'CDM' | 'CM' | 'CAM'
    | 'LM' | 'RM' | 'LW' | 'RW' | 'ST';

type Role = 'DEFENSIVE' | 'NORMAL' | 'OFFENSIVE' | 'CREATOR' | 'PRESSING';

interface Player {
    id: string;
    name: string;
    nation: string;
    primaryPosition: Position;
    alternativePositions: Position[];
    stats: {
        speed: number;
        shooting: number;
        passing: number;
        physical: number;
        technique: number;
        intelligence: number;
    };
};

interface FormationSlot {
    id: string;
    expectedPosition: Position;
    anchor: { x: number; y: number };
    neighbors: string[];
}

interface LineupSlot {
    formationSlotId: string;
    playerId: string;
    role: Role;
}

interface TeamSheet {
    formationId: string;
    starters: LineupSlot[];
    bench: string[];
}
```

```

collectiveTactics: {
  blockHeight: 'LOW' | 'NORMAL' | 'HIGH';
  buildUp: 'SHORT' | 'BALANCED' | 'DIRECT';
};
substitutionRules: SubstitutionRule[];
}

```

Annexe C — Utilities de départ

Action	Variables principales de utility
Tir	angle, distance, espace, score, poste, rôle, pression
Passe courte	sécurité, disponibilité cible, progression, tactique
Passe longue	progression, espace cible, risque, construction directe
Dribble	espace devant, adversaire, Technique, progression attendue
Avancer	espace, ancre, phase collective
Temporiser	pression faible, avantage au score, absence d'option
Pressing	distance ballon, rôle, hauteur bloc, énergie
Interceptor	trajectoire probable, distance, Intelligence
Tacler	proximité, urgence, risque de faute
Remplacement	écart à l'ancre, phase, danger local

Annexe D — Paramètres à calibrer en priorité

- Durée logique du match.
- Températures minimale/maximale et gamma d'Intelligence.
- Poids des utilities d'action.
- Courbes distance/angle pour les tirs.
- Probabilités de passe selon distance et pression.
- Poids des duels et interceptions.
- Vitesse de déplacement avec/sans ballon.
- Coûts de fatigue et effet de l'énergie.
- Bonus de synergie et cap.
- Matrice de compatibilité hors poste.
- Probabilités de faute, carton et blessure.
- Seuils et priorités de remplacement.

Annexe E — Décisions de design figées en Version 1.0

Sujet	Décision
Simulation	Pré-simulée côté serveur, seed déterministe.
Durée	90 min affichées ; 360 s logiques par défaut ; configurable.
Stats	Vitesse, Tir, Passe, Physique, Technique, Intelligence.
Clubs	Ignorés pour le gameplay initial.

Sujet	Décision
Nationalité	Utilisée pour synergie locale entre voisins tactiques.
Hors poste	Malus principalement sur Intelligence.
Match	Aucune intervention du coach en direct.
Rôles	Défensif, Normal, Offensif, Créateur, Pressing.
Collectif	Hauteur de bloc + style de construction.
Fatigue	Intra-match uniquement, reset après match.
Remplacements	5 maximum, automatisés par règles pré-match.
Cartons	Pas d'accumulation de jaunes entre matchs ; rouge = suspension 1 match.
Blessures	Modèle MVP : indisponibilité du prochain match uniquement.
Mode	Moteur d'abord ; cible finale navigateur, bots + PvP asynchrone.

Annexe F — Références de données initiales

Pour le prototype, deux types de datasets récents sont adaptés : un dataset FC 26 complet exposant notamment Position, Alternative positions, Nation, les sous-attributs de rythme/tir/passe/dribble/défense/physique et les attributs GK ; ou un dataset issu de SoFIFA disposant d'un schéma équivalent. Le moteur doit rester indépendant de la source : seul le script d'import connaît les noms de colonnes externes.

Candidat pratique : « EAFC26 Player Database » (Kaggle, Aiden Flynn), dont l'en-tête public inclut les colonnes nécessaires à la formule proposée.

Alternative : « FC 26 (FIFA 26) Player Data » (Kaggle, rovez), données détaillées issues de SoFIFA.

Les questions de licence et d'exploitation commerciale sont volontairement hors du périmètre fonctionnel de cette version du document.